

NABENDU DAS

GAME DEVELOPER | GAMEPLAY PROGRAMMER

CONTACT

✉ nabendudas2001@gmail.com
☎ +91-9475563268
📍 West Bengal, India
🌐 [Portfolio Website](#)
🐙 github.com/Nandy1002
🌐 linkedin.com/in/nabendu-das-1790a8202

SKILLS

Game Engines

Unity 3D & 2D, Unreal Engine 5

Programming & Concepts

C#, C++, Blueprints, OOP, SOLID, Design Patterns, DSA, Memory Optimization, Clean Code

Unity Stack

ScriptableObject Architecture, UGUI & UI Toolkit, New Input System, Cinemachine & Timeline, URP, Coroutines & Async, NavMesh AI, DOTween

Unreal Stack

Multiplayer Replication (RPCs, RepNotifies), Game Framework, UMG UI, Online Subsystems (Sessions), State Trees / AI, Animation Blueprints

Core Development

Gameplay Systems, Multiplayer FPS, UI/FTUE, Game Feel, Rapid Prototyping

Tools & Secondary

Git, Plastic SCM, Visual Studio, Python, GitHub Copilot

EDUCATION

Master of Computer Applications

RV College of Engineering, Bangalore
Sep 2022 – Sep 2024 · CGPA 8.83

B.Sc. Mathematics

The University of Burdwan, WB
Jul 2019 – Aug 2022 · CGPA 8.89

CERTIFICATION

Soft Skills & Personality

NPTEL

SUMMARY

Results-driven Gameplay Programmer with 2 years of professional experience engineering **30+ mobile game prototypes and live projects** in Unity (C#). Demonstrated expertise in Object-Oriented Programming (OOP) and component-based architecture, successfully driving mechanics from concept to deployment. Adept at system design, performance optimization, and cross-functional collaboration. Recently expanded technical repertoire into Unreal Engine 5 (C++ and Blueprints) to architect networked multiplayer experiences.

EXPERIENCE

Unity Game Developer

Feb 2025 – Present

Zapplogics Solution · Remote

- Engineered **30+ mobile prototypes and live titles**, independently directing core gameplay loops and rapid validation phases to meet aggressive market deadlines.
- Architected highly reusable component-based gameplay systems, reducing average feature integration time by 30% across concurrent project lifecycles.
- Orchestrated the design of scalable UI architectures and FTUE systems, significantly improving player onboarding and engagement metrics.
- Spearheaded comprehensive performance profiling initiatives, achieving a stable 60 FPS baseline on low-end Android hardware while minimizing memory overhead.

AR/VR Developer Intern

Aug 2023 – Dec 2023

Samsung Prism · Bangalore

- Programmed a **multi-user VR simulation** featuring real-time avatar synchronization and dynamic positioning algorithms, successfully supporting concurrent user sessions.
- Iterated on interactive spatial interfaces and player comfort mechanics, consistently delivering stable milestones on schedule within a strict research timeline.

Salesforce Support / Release Intern

Feb 2024 – Jul 2024

Volvo Car · Bangalore

- Streamlined enterprise release workflows, delivering a 15% increase in deployment efficiency while managing access control protocols for cross-functional teams.

PROJECTS

Capture The Flag (Unreal Engine 5, C++, Blueprints) — Multiplayer FPS

Architected a networked multiplayer FPS featuring a custom C++ lobby, session hosting, and dynamic team balancing. Implemented a replicated client-server flag framework (pick, carry, return) ensuring 100% state synchronization, and designed custom GameMode classes to manage server-authoritative scoring and UMG HUD updates.

TicTacTale (Unity 6, C#) — Solo Turn-Based Duel Game

Engineered a scalable N×N board architecture incorporating dynamic win detection and a coroutine-driven AI opponent. Enhanced player feedback using DOTween to implement hitstop and squash-and-stretch animations, elevating overall user engagement.

The Cursed Chaos (Unity 6) — Horror Prototype

Programmed a comprehensive first-person interaction and modular inventory system utilizing ScriptableObjects for data-driven design. Integrated NavMesh-driven AI pursuit behaviors alongside Timeline and Cinemachine to construct seamless narrative sequences.

FPS Inventory System (Unity, C#) — Gameplay Architecture

Constructed a highly modular inventory framework across 12 decoupled classes, managing raycast detection and UI-separated slot behavior to eliminate monolithic dependencies and improve code maintainability.